

IXChange BV

SDR-SLC

VITA-SLC I/Q Stream Transport Specification

Field	Value
Document	SDR-SLC-VITA-1.0
Title	VITA-SLC I/Q Stream Transport Specification
Version	Protocol 1.0 Release 1.2 (VITA-49.0 alignment, transmit profile)
Profile Names	VITA-A/1.0 (IF Data) VITA-A/1.0-CTX (Context) VITA-A/1.0-TX (Transmit IF Data)
Status	RELEASE CANDIDATE
Date	September 2026
Author	Ivo van Ling, PA2IX
Organisation	IXChange BV
Licence	Creative Commons CC BY 4.0 — Open Source

Copyright © 2026 IXChange BV. All rights reserved. Published under Creative Commons CC BY 4.0.

Table of Contents

Revision History	4
1. Introduction	4
1.1 What changed in Release 1.2, and why.....	5
2. Normative Language	5
3. Terminology	5
4. Implementing VITA-SLC from Scratch	6
4.1 What You Are Building.....	6
4.2 Tier 1 Minimum: What You Must Send.....	7
4.3 What u8 Encoding Means.....	7
4.4 Tier 2: Adding the Context Packet.....	8
4.5 The Reference Level: Why It Matters.....	8
4.6 The AGC Problem.....	9
4.7 Timestamps on a device without a disciplined clock.....	9
4.8 Testing Your Implementation.....	9
5. UDP Transport	10
6. VITA-A/1.0 IF Data Packet Format [RX-T1]	10
6.1 VRT Header Fields.....	10
6.2 Stream Identifier.....	11
6.3 Class Identifier.....	11
6.4 Timestamps.....	11
6.5 I/Q Payload Encodings.....	12
7. VITA-A/1.0-CTX Context Packet Profile [RX-T2]	13
7.1 Context Packet Layout.....	13
7.2 VRT Header for Context Packets.....	13
7.3 Context Indicator Field (CIF0).....	14
7.4 Reference Level Field.....	14
7.5 Payload Format Field.....	15
7.6 Gain Tracking: When to Send Context Packets.....	15
7.7 Receiver Behaviour (Client Rules).....	16
7.8 Complete Annotated Packet Examples.....	16
7A. VITA-A/1.0-TX Transmit IF Data Profile [TX-T1 accept • TX-T2 send]	17
7A.1 Packet Layout.....	17
7A.2 The timestamp is a sample count, not a time.....	18
7A.3 Obtaining the mapping.....	18
7A.4 Burst delimiting.....	18
7A.5 Payload.....	19
7A.6 Accepting the receive profile on the transmit port [TX-T1].....	19
7A.7 Data timeout.....	19
7A.8 Worked example.....	19
7A.9 Client conformance checklist.....	19
8. I/Q Format Negotiation	20
9. Stream Identification and Multiplexing	20
10. Clock and Timestamp Model	21
10.1 PTP Integration.....	21
11. Multi-Stream Operation	21
12. Conformance Requirements	21
12.1 IF Data Packet Sender [RX-T1].....	21
12.2 Context Packet Sender [RX-T2].....	22
12.3 Transmit Receiver [TX-T1] and Burst Receiver [TX-T2].....	22
12.4 Client Requirements.....	22
13. Security Considerations	23
14. Forward Compatibility	23
15. References	23
15.1 Normative References.....	23
15.2 Informative References.....	24
Appendix A. Packet Size Reference Table	24

Appendix B. Profile Comparison.....24

Appendix C. Implementation Checklist.....25

Revision History

Rev	Date	Author	Description
1.0 draft	2026-05-01	I. van Ling	Initial release. VITA-A/1.0 IF Data profile. u8 and int16 encodings, UDP transport, PTPv2/GPS timestamps.
1.1 draft	2026-05-21	I. van Ling	Added VITA-A/1.0-CTX Context packet profile with Payload Format field for signal type.
1.2 draft	2026-05-22	I. van Ling	Extended VITA-A/1.0-CTX with Reference Level field. Gain-tracking rules and power calculation formula.
1.0 release	2026-05-22	I. van Ling	Consolidated release. All drafts merged.
1.1 release	2026-05-23	I. van Ling	Revised for implementers. Added §4 'Implementing VITA-SLC from Scratch'. Implementation tier markers, annotated examples, STM32 and Raspberry Pi walkthrough.
1.2 release	2026-09-14	I. van Ling	VITA-49.0 alignment. The Context packet is corrected to the field positions and encodings VITA-49.0 actually defines: CIFO bits 24 and 15, Reference Level as Q9.7 in the low half-word, Payload Format computed from the negotiated encoding. New VITA-A/1.0-TX transmit profile with sample-count timestamps and burst delimiters. Clock rule aligned with SDR-SLC-CP-1.0 Release 1.3. Control-plane field paths updated to Release 1.2/1.3. Tier markers per function block. Word-alignment rule for u8 payloads. Class Identifier semantics corrected. Editorial corrections.
1.2 release	2026-09-16	I. van Ling	Amendment folded into Release 1.2 before publication. <code>zero_point</code> for unsigned encodings: a device that knows its converter's real midpoint declares it on the format entry (§6.5, §8); clients subtract it before scaling and add it before packing. Default $2^{(\text{bits}-1)}$.

WARNING	Release 1.2 changes three words of the Context packet on the wire and is not compatible with Context packets built against Release 1.1. IF Data packets are unchanged. See §1.1 for the reasoning. This document remains a RELEASE CANDIDATE; Protocol 1.0 has never been declared final.
----------------	---

1. Introduction

VITA-SLC is the I/Q stream transport layer of the SDR-SLC protocol suite. It defines exactly how raw I/Q sample data travels between the radio hardware and the host computer over UDP. This document specifies three packet profiles: VITA-A/1.0 for receive sample data, VITA-A/1.0-CTX for metadata that makes the receive stream self-describing, and VITA-A/1.0-TX for transmit sample data, including bursts scheduled on an exact sample.

The profiles are constrained subsets of the VITA-49.0 standard (the VITA Radio Transport specification). VITA-49 is a well-established open standard used in professional and military radio systems. By building on it rather than inventing a custom binary format, SDR-SLC packets can be examined with existing VITA-49 analysis tools, and the format is documented independently of this project.

That promise is only worth something if the subset really is a subset. Release 1.1 of this document was not: its Context packet placed fields in CIFO bits that VITA-49.0 reserves, encoded the Reference Level in a format the standard does not use, and carried a Payload Format word that did not describe the payload. Every implementer who checked the standard found this, and each found it separately. Release 1.2 corrects all three so that a generic VITA-49 tool decodes an SDR-SLC Context packet as what it is.

The constraint in VITA-A/1.0 is deliberate. Full VITA-49 has many optional features that add implementation complexity without benefiting a simple SDR system. VITA-A/1.0 fixes the options that matter (timestamping mode, class identifier, no trailer) and leaves the I/Q encoding negotiable. The result is a format that is simple enough to implement on a bare STM32H7 in an afternoon, yet rich enough to support calibrated power display, direct sampling mode, late-joining receivers, and sample-accurate transmit bursts for TDMA digital voice.

This document is both a normative specification and a practical guide. It explains not just what the packet format is, but why each field exists and how to implement it correctly on real hardware.

NOTE	This document covers the wire format only. Stream lifecycle management (open, start, stop, close) and format
-------------	--

	negotiation are specified in SDR-SLC-CP-1.0 Release 1.3. The SLC-CP control plane negotiates the I/Q encoding and stream parameters before UDP data begins flowing. Read SDR-SLC-CP-1.0 first if you are new to the protocol suite.
--	---

1.1 What changed in Release 1.2, and why

Three things in the Context packet are different on the wire, and one profile is new.

- CIFO is 0x01008000 (with bit 31 set on change), not 0x00000082. VITA-49.0 assigns Reference Level to bit 24 and the Data Packet Payload Format to bit 15; bits 7 and 1 are reserved. The field order is unchanged, because it follows descending bit order in both cases.
- The Reference Level is a signed 16-bit value in the low half of the word with seven fractional bits (Q9.7, 1/128 dBm), and the high half is zero. That is VITA-49.0 §7.1.5.9. Release 1.1 described a 16.16 encoding under the same section number; the section does not say that.
- The Payload Format word 0 is computed from the negotiated I/Q encoding, so that it describes the payload. Release 1.1 used one constant regardless of encoding, and the constant did not decode to what its own annotation claimed.
- VITA-A/1.0-TX is new. Release 1.1 had no transmit direction at all; SDR-SLC-CP-1.0 has had a transmit block since Release 1.1 and both reference implementations already speak this profile.

The reference implementation `sdr-slc-rtld` had, by accident of following VITA-49.0 rather than this document, two of the three Context fields right. Nothing in this release asks any implementation to move away from the standard.

2. Normative Language

Keyword	Meaning
SHALL	Absolute requirement. Non-conformant if omitted.
SHALL NOT	Absolute prohibition.
SHOULD	Strong recommendation. Deviation requires justification.
MAY	Optional. Implementations may include or omit.

Tier markers appear throughout this document. They use the per-function-block tiers of SDR-SLC-CP-1.0 Release 1.3:

Marker	Meaning in this document
[RX-T1]	Required of every device implementing the rx block. IF Data packets.
[RX-T2]	Required of an rx block at Tier 2 and above. Context packets.
[TX-T1]	Required of every device implementing the tx block. Continuous transmit.
[TX-T2]	Required of a tx block at Tier 2 and above. Scheduled bursts.

3. Terminology

Term	Definition
IF Data Packet	A VITA-49 packet that carries I/Q sample payload. Packet Type 0x1 with Stream ID. Defined by the VITA-A/1.0 profile (receive) and the VITA-A/1.0-TX profile (transmit).
Context Packet	A VITA-49 IF Context packet that carries stream metadata (signal type and Reference Level). Packet Type 0x4 with Stream ID. Defined by the VITA-A/1.0-CTX profile.
VITA-A/1.0	The SDR-SLC constrained profile for receive IF Data packets. The minimal transmission format.
VITA-A/1.0-CTX	The SDR-SLC constrained profile for Context packets. Required at rx Tier 2.
VITA-A/1.0-TX	The SDR-SLC constrained profile for transmit IF Data packets: sample-count timestamp and burst delimiters. Required at tx Tier 2; a tx Tier 1 device accepts plain VITA-A/1.0 instead.
Stream ID	A 32-bit integer assigned by the device that uniquely identifies a stream within a TCP session.

Term	Definition
	Shared between IF Data packets and their associated Context packets. Distinct from the SLC-CP string <code>stream_id</code> .
VRT Header	The first 32-bit word of every VITA-49 packet. Contains packet type, flags, timestamp mode indicators, packet counter, and total size.
CIFO	Context Indicator Field 0. A 32-bit bitmask present in every Context packet that tells the receiver which context fields follow in the packet body.
Payload Format	The VITA-49.0 §7.1.5.18 Data Packet Payload Format field: two words describing the signal type and sample encoding in the associated IF Data stream.
Reference Level	The received signal power at the antenna port that corresponds to a full-scale I/Q sample, in dBm. Enables calibrated power display.
dBFS	Decibels relative to full scale. 0 dBFS is the maximum representable amplitude (32767 for int16).
TSI	Timestamp Integer mode bits in the VRT header. 01b = UTC seconds; 11b = other (relative); 00b = none.
TSF	Timestamp Fractional mode bits. 10b = real time (picoseconds); 01b = sample count; 00b = none.
OUI	Organizationally Unique Identifier. IXChange BV: 0x402814. Embedded in the VITA-49 Class Identifier field of every IF Data packet, because IXChange BV defined the packet class.
PCC	Packet Class Code. 0x0001 for VITA-A/1.0, 0x0002 for VITA-A/1.0-TX.
ICC	Information Class Code. Fixed at 0x0000 in all SDR-SLC profiles.
<code>samples_per_packet</code>	Number of I/Q pairs per UDP packet. Fixed for the lifetime of a receive stream; advisory for a transmit stream. Returned in the SLC-CP <code>open_stream</code> response.
<code>sample_count</code>	Running count of I/Q pairs output since the most recent <code>start_rx</code> . Used to derive the fractional timestamp of receive packets.
Sample counter	The device's free-running count of converted samples, per direction, read with SLC-CP <code>get_timestamps</code> . The domain of VITA-A/1.0-TX timestamps.
Burst	A bounded transmission with a defined start sample and end, delimited by trailer indicator bits in VITA-A/1.0-TX.
native format	The I/Q encoding that the hardware ADC or DAC produces or consumes directly. Marked <code>native:true</code> in <code>capabilities.iq_formats</code> .

4. Implementing VITA-SLC from Scratch

This chapter is for the experimenter who has never worked with VITA-49 before and wants to understand what they actually need to send. It walks through the complete picture from wire bits to working stream before getting into field-level detail.

4.1 What You Are Building

Your device receives a `start_rx` command over the SLC-CP TCP connection. From that moment, it must start sending UDP packets to the IP address and port the client specified in `open_stream`. Each UDP packet is a VITA-A/1.0 IF Data packet. The payload of each packet is a block of raw I/Q sample pairs.

That is the core of it. The rest — the VRT header, the Class Identifier, the timestamps — is framing that tells the receiver how to interpret the payload and when the samples were captured.

What a VITA-A/1.0 packet looks like on the wire:

```
[Ethernet frame][IP header][UDP header]
[4 bytes: VRT Header      ] -- packet type, flags, size, counter
[4 bytes: Stream Identifier] -- which stream this belongs to
[4 bytes: Class ID Word 0  ] -- 0x00 | OUI 0x402814
[4 bytes: Class ID Word 1  ] -- ICC 0x0000 | PCC 0x0001
[4 bytes: Timestamp Integer] -- UTC seconds (32-bit)
[4 bytes: Timestamp Frac High] -- picoseconds, bits 63:32
[4 bytes: Timestamp Frac Low ] -- picoseconds, bits 31:0
[N*4 bytes: I/Q Payload    ] -- N IQ pairs, interleaved
```

Total header: 28 bytes.

With 360 IQ pairs at int16: payload = 1440 bytes.
 Total IP datagram: 28 + 1440 + 8 (UDP) + 20 (IP) = 1496 bytes.
 Ethernet MTU limit: 1500 bytes. We have 4 bytes of margin.

4.2 Tier 1 Minimum: What You Must Send

At rx Tier 1, you must send correctly formatted IF Data packets. Context packets are not required. The client will assume complex I/Q mode and use any Reference Level it obtained from SLC-CP capabilities.

```
// C pseudocode for building a minimal VITA-A/1.0 IF Data packet
// Assumes 360 IQ pairs, int16 encoding, big-endian output.

uint32_t stream_id    = 1;      // vita.stream_id from open_stream response
uint32_t packet_count = 0;      // 0..15, increment each packet
uint64_t sample_count = 0;      // increment by 360 each packet
uint32_t fs_hz        = 2048000;

void build_packet(uint8_t *buf, int16_t *iq_samples) {
    // VRT Header constant 0x18600000:
    // [31:28]=0001b IF Data + Stream ID [27]=1 Class ID present
    // [26]=0 no trailer [25:24]=00b [23:22]=01b TSI UTC
    // [21:20]=10b TSF picoseconds
    uint32_t pkt_size = 7 + 360; // 7 header words + 360 payload words
    uint32_t hdr = 0x18600000 | ((packet_count & 0xF) << 16) | pkt_size;
    write_be32(buf+0,  hdr);
    write_be32(buf+4,  stream_id);
    write_be32(buf+8,  0x00402814); // OUI
    write_be32(buf+12, 0x00000001); // ICC=0, PCC=1
    write_be32(buf+16, utc_seconds_now()); // integer timestamp, see section 10

    // Fractional timestamp in picoseconds, integer arithmetic only
    uint64_t picos = (sample_count % fs_hz) * 1000000000000ULL / fs_hz;
    write_be32(buf+20, (uint32_t)(picos >> 32));
    write_be32(buf+24, (uint32_t)(picos & 0xFFFFFFFF));

    // I/Q payload: 360 pairs, each pair is I (int16) then Q (int16), big-endian
    for (int i = 0; i < 360; i++) {
        write_be16(buf+28 + i*4, iq_samples[i*2]); // I
        write_be16(buf+28 + i*4 + 2, iq_samples[i*2+1]); // Q
    }
    packet_count = (packet_count + 1) & 0xF;
    sample_count += 360;
}
```

TIP

The VRT header value 0x18600000 is a constant for VITA-A/1.0 if you exclude the packet counter and size fields. Compute it once. The only things that change packet-to-packet are the counter (bits 19:16), the packet size (bits 15:0), and the payload content.

4.3 What u8 Encoding Means

The RTL-SDR natively produces 8-bit unsigned offset binary samples. Each I or Q component is a byte running from 0 (maximum negative) to 255 (maximum positive). The nominal midpoint — the value that means "no signal" — is 128, but a real converter's midpoint rarely is: the RTL2832U sits at about 127.4, and treating it as 128 puts a DC spike in the middle of every spectrum display. A device that knows its midpoint declares it as `zero_point` on the format entry (§6.5, §8), and the client subtracts that instead of 128.

```
// u8 encoding: 8 bits per I or Q component, 2 bytes per IQ pair
// Value 0 = maximum negative Value 255 = maximum positive
// zero signal = zero_point (nominal 128; RTL2832U ~127.4)
// Relationship to int16 (nominal midpoint):
// u8 = (int16_sample >> 8) ^ 0x80
// int16 = ((int32_t)(u8 ^ 0x80)) << 8
// Payload for u8: [I_u8][Q_u8][I_u8][Q_u8]...
// Packet size with 360 pairs at u8:
// payload = 360 * 2 = 720 bytes = 180 words
// total datagram = 720 + 28 + 8 + 20 = 776 bytes
```

Because the Packet Size field counts 32-bit words, a u8 payload SHALL contain an even number of pairs (\$6.5).

TIP	If you are bridging an RTL-SDR, use u8 encoding. It is the native format, requires no conversion, and halves the UDP bandwidth compared to int16. Declare it in <code>capabilities.rx.iq_formats</code> with <code>native:true</code> . The client will request it by default. Measure the midpoint once with the antenna terminated — average a few seconds of raw I and Q — and declare it as <code>zero_point</code> ; <code>sdr-slc-rtld</code> defaults to 127.4 and lets the operator override it.
------------	--

4.4 Tier 2: Adding the Context Packet

At rx Tier 2, your device must send one Context packet as the very first UDP packet after `start_rx`, before any IF Data packets. You must also send a new Context packet whenever the gain changes or the signal mode changes.

The Context packet is 24 bytes. It tells the client two things: what kind of signal is in the payload (complex I/Q or real) and how it is encoded, and what signal power at the antenna corresponds to a full-scale sample (the Reference Level).

```
// C pseudocode for building a VITA-A/1.0-CTX Context packet.
// pf_word0 is the Payload Format constant for the negotiated encoding,
// taken from the table in section 7.5. For int16 complex: 0x200003CF.

void build_context_packet(uint8_t *buf, uint32_t stream_id,
                        float ref_level_dbm, uint32_t pf_word0,
                        bool changed) {
    // VRT Header: type 0100b (IF Context + Stream ID), no CID,
    // TSI=00b, TSF=00b, size = 6 words
    write_be32(buf+0, 0x40000006 | ((ctx_count & 0xF) << 16));
    write_be32(buf+4, stream_id);

    // CIF0: bit 24 Reference Level, bit 15 Payload Format,
    // bit 31 Context Field Change Indicator when something changed
    uint32_t cif0 = 0x01008000 | (changed ? 0x80000000 : 0);
    write_be32(buf+8, cif0);

    // Reference Level: VITA-49.0 7.1.5.9 -- signed 09.7 dBm in bits 15:0,
    // bits 31:16 zero. -30.0 dBm -> -3840 -> 0xF100 -> 0x0000F100
    int16_t rl_q7 = (int16_t)lroundf(ref_level_dbm * 128.0f);
    write_be32(buf+12, (uint32_t)(uint16_t)rl_q7);

    // Payload Format: word 0 from the table, word 1 always zero
    write_be32(buf+16, pf_word0);
    write_be32(buf+20, 0x00000000);

    ctx_count = (ctx_count + 1) & 0xF;
}

// When to call it:
// After start_rx:          build_context_packet(..., changed=true)
// After set_control rf_gain: build_context_packet(..., changed=true)
// After set_control direct_sampling: pf_word0 for the real variant
```

TIP	An RTL-SDR bridge has two possible payloads (u8 complex, u8 real) and therefore exactly two Payload Format constants: <code>0x300001C7</code> and <code>0x100001C7</code> . Store them and select on the <code>direct_sampling</code> state. A device that offers several encodings selects the constant when the stream is opened.
------------	---

4.5 The Reference Level: Why It Matters

Without the Reference Level, the client's waterfall S-meter shows relative levels only. With the Reference Level, the client computes:

```
P_dBm = ReferenceLevel_dBm + 20 * log10(RMS_amplitude / full_scale)

Example with Reference Level = -30.0 dBm and u8 encoding (full_scale = 127):
Measured RMS    = 12.7 counts = 0.1 of full scale
Level in dBFS   = 20 * log10(0.1) = -20.0 dBFS
P_dBm           = -30.0 + (-20.0) = -50.0 dBm

The Reference Level changes with gain:
```

At `rf_gain = 20 dB`, Reference Level = `-30.0 dBm`
 Operator increases `rf_gain` to `30 dB (+10 dB gain)`
 New Reference Level = `-30.0 - 10.0 = -40.0 dBm`

TIP

For an uncalibrated RTL-SDR dongle, initialise the Reference Level to a nominal value like `-30.0 dBm` at the preferred gain setting and publish `reference_level_accuracy_db: 8` in capabilities. The absolute accuracy will be poor but the tracking across gain changes will be correct as long as you subtract the gain delta each time.

4.6 The AGC Problem

When the RTL-SDR hardware AGC is active, the gain changes autonomously without a `set_control` command from the client. The device must still emit a Context packet whenever the AGC changes the gain. This is the only case where Context packets are generated by hardware events rather than client commands.

NOTE

Rate-limit AGC Context packet emission to no more than one per 100 ms. Some RTL-SDR tuners can update AGC gain tens of times per second. Keep a timestamp of the last Context packet and skip emission if the previous one was less than 100 ms ago; send one packet at the end of the quiet period instead.

4.7 Timestamps on a device without a disciplined clock

Your device needs to fill in the Timestamp Integer field (seconds) and the Timestamp Fractional field (picoseconds). The fractional field always comes from the sample count (§6.4.2). The integer field depends on what time source you have:

- Host clock only (a Raspberry Pi or a PC running NTP, no PTP or GPS): stamp your best estimate of UTC under `TSI=01b`, and report `source: "free_running"`, `synced: false` in `get_clock_status`. This is what both reference daemons do. The client can still align two such receivers to within host-clock accuracy, which is useful.
- PTP grandmaster on the LAN: discipline your clock to it, stamp UTC under `TSI=01b`, report `source: "ptpv2"`, `synced: true`. Timestamps are then accurate to better than 100 μ s.
- No time source at all (bare STM32 with no RTC and no network time): count seconds from `start_rx` under `TSI=11b` ("Other"). The client treats the value as relative. Do not stamp a fake UTC under `TSI=01b`; the mode bits are how the client knows what it is looking at.

TIP

For a first implementation on Linux, `clock_gettime(CLOCK_REALTIME)` at the first sample of each `start_rx`, then derive everything else from the sample count. That is all `sdr-slc-rtd` does.

4.8 Testing Your Implementation

- Send SLC-CP `open_stream` and `start_rx` from a minimal test client (netcat plus a Python script works fine).
- Capture UDP packets on the client host with Wireshark, filtering on the destination port. Verify: packet arrives, size is correct (776 bytes for `u8/360` pairs, 1496 bytes for `int16/360` pairs), source IP matches the device.
- Decode the first 28 bytes manually against the field definitions in §6. Verify the VRT Header has `type=0001b`, `CID=1`, `TSI=01b`, `TSF=10b`. Verify `OUI = 0x402814`.
- Feed the UDP stream into the SDR-SLC client. You should see a waterfall. If the waterfall looks like noise but responds to signals, your packet format is correct. If it looks completely wrong, check byte order (everything must be big-endian).
- Verify the Packet Count field increments correctly. The client will report dropped packets if it sees gaps in the 0-15 sequence.
- If you send Context packets, decode one with a generic VITA-49 dissector (Wireshark has one). It should show a Reference Level in dBm and a Payload Format of the right item size. If it does not, you have Release 1.1 constants somewhere.

5. UDP Transport

VITA-SLC packets travel over UDP/IPv4. The choice of UDP over TCP is deliberate: UDP has lower overhead, no retransmission delay, and allows the receiver to detect and measure packet loss without waiting for timeouts.

Parameter	Value / Rule
Protocol	UDP/IPv4. Unicast only. Multicast is not supported.
Receive stream, source	Device IP and port, returned by <code>open_stream</code> as <code>vita.source</code> . The port is advertised as <code>source_port</code> in <code>capabilities.rx.transports[]</code> ; default 4621.
Receive stream, destination	Client IP and port, specified by the client in <code>open_stream</code> as <code>vita.destination</code> . Chosen by the client; the device never assumes it.
Transmit stream, destination	Device IP and <code>listen_port</code> , returned by <code>open_stream</code> as <code>vita.listen_port</code> and advertised in <code>capabilities.tx.transports[]</code> ; default 4622.
Transmit stream, source	Client IP and port. The client MAY declare them in <code>open_stream</code> as <code>vita.source</code> ; a device SHOULD then accept transmit data from that address only.
MTU constraint	Each IF Data IP datagram SHALL NOT exceed 1500 bytes. Context packets are 24 bytes of payload; total datagram 52 bytes.
Packet rate	<code>sample_rate_sps / samples_per_packet</code> packets per second. E.g. 2,048,000 / 360 ≈ 5,689 packets/second.
Jitter	The sender SHOULD pace packets at the hardware sample rate. Bursting multiple packets then pausing degrades receiver buffer performance.

TIP	On STM32H7 with lwIP and FreeRTOS: use a DMA-driven timer to trigger packet assembly and send at the correct packet rate. Do not use a polling loop. Pre-allocate a pbuf for each packet and reuse it by resetting the payload pointer each cycle.
------------	--

6. VITA-A/1.0 IF Data Packet Format [RX-T1]

Every UDP payload sent by a VITA-A/1.0 device is a VITA-A/1.0 IF Data packet. The format is fixed: 28 bytes of header followed by the I/Q payload. There is no trailer.

Figure 1 -- Complete VITA-A/1.0 IF Data Packet Layout

Offset	Size	Field
+0	4 bytes	VRT Header
		[31:28]=0001b Packet Type: IF Data + Stream ID
		[27]=1 Class ID Present (CID)
		[26]=0 No Trailer
		[25:24]=00b Reserved (always 0)
		[23:22]=01b TSI: UTC integer seconds (11b: relative)
		[21:20]=10b TSF: picoseconds
		[19:16]=0..F Packet Count (modulo 16)
		[15:0]=size Packet Size in 32-bit words
+4	4 bytes	Stream Identifier (32-bit, assigned by device)
+8	4 bytes	Class ID Word 0: [0x00 reserved OUI 0x402814]
+12	4 bytes	Class ID Word 1: [ICC 0x0000 PCC 0x0001]
+16	4 bytes	Timestamp Integer: UTC seconds since epoch (32-bit)
+20	4 bytes	Timestamp Fractional HIGH (bits 63:32)
+24	4 bytes	Timestamp Fractional LOW (bits 31:0)
+28	N x 4 B	I/Q Payload: N = <code>samples_per_packet</code> IQ pairs, interleaved I0 Q0 I1 Q1 ... in the negotiated encoding.

Total fixed header: 28 bytes (7 words).

Example: 360 pairs, int16 -> payload 1440 B -> datagram 1496 B.

Example: 360 pairs, u8 -> payload 720 B -> datagram 776 B.

6.1 VRT Header Fields

Bits	Field	Value	Notes
[31:28]	Packet Type	0001b	IF Data Packet with Stream ID.

Bits	Field	Value	Notes
[27]	Class ID Present	1	The Class Identifier fields at +8 and +12 are present. Always 1.
[26]	Trailer Present	0	No trailer in the receive profile. A receiver that sees 1 here on a receive stream SHALL discard the packet.
[25:24]	Reserved	00b	Reserved by VITA-49.0. Transmitters set 0; receivers ignore. (VITA-49.2 assigns meaning to these bits; SDR-SLC profiles do not use it.)
[23:22]	TSI	01b or 11b	01b: UTC seconds since the Unix epoch. 11b: relative seconds since <code>start_rx</code> , only on a device with no time source (§6.4.1).
[21:20]	TSF	10b	Real time: picoseconds, 64-bit unsigned.
[19:16]	Packet Count	0..15	Modulo-16 counter per stream, per packet type. Increments with each IF Data packet. Clients use gaps to detect dropped packets.
[15:0]	Packet Size	Variable	Total packet size in 32-bit words, counting every word from the VRT Header to the end of the payload. int16, 360 pairs: $7 + 360 = 367$ (0x016F). u8, 360 pairs: $7 + 180 = 187$ (0x00BB).

6.2 Stream Identifier

A 32-bit word immediately follows the VRT Header. Its value is the VITA-49 numeric Stream ID, returned by SLC-CP in the `open_stream` response as `vita.stream_id`. The device assigns this value and it does not change for the lifetime of the stream session.

- The Stream ID SHALL NOT be reused within a TCP control session. If you open a new stream after closing the previous one, assign a new Stream ID.
- The Stream ID is opaque to the client. Assign sequential integers starting from 1, or use a random value.
- When multiple streams are active simultaneously (an RX stream and a TX stream), each has its own Stream ID.

6.3 Class Identifier

Two 32-bit words at offsets +8 and +12 form the VITA-49 Class Identifier. This pair of values globally and unambiguously identifies every VITA-A/1.0 packet on any network.

Word	Bits	Field	Value	Description
Word 0	[31:24]	Reserved	0x00	Always zero per VITA-49.0.
Word 0	[23:0]	OUI	0x402814	IXChange BV IEEE-registered OUI. Big-endian bytes 0x40, 0x28, 0x14.
Word 1	[31:16]	ICC	0x0000	Information Class Code. Always 0x0000.
Word 1	[15:0]	PCC	0x0001	Packet Class Code. 0x0001 identifies the VITA-A/1.0 receive IF Data profile. 0x0002 identifies VITA-A/1.0-TX (§7A).

In VITA-49, the OUI in a Class Identifier names the organisation that defined the packet class, not the organisation that built the device. Every VITA-A/1.0 packet carries 0x402814 whoever built the hardware, because IXChange BV defined the profile. A third party defining its own, different profile uses its own OUI, and an SDR-SLC receiver correctly rejects those packets as not VITA-A/1.0.

WARNING	Receivers SHALL reject IF Data packets where OUI $\neq$ 0x402814 or PCC is not a value defined by this document. Do not use the placeholder 0x000000 OUI. Do not substitute your own OUI in a VITA-A/1.0 packet: it would make your device invisible to every SDR-SLC client.
----------------	---

6.4 Timestamps

6.4.1 Timestamp Integer

A 32-bit unsigned integer at offset +16. Under TSI=01b it contains the number of whole seconds elapsed since the Unix epoch (1970-01-01T00:00:00Z). This field overflows on 2106-02-07; the 32-bit limit is inherited from VITA-49.0.

A device SHALL stamp its best estimate of UTC under TSI=01b whenever it has any time source at all, including an undisciplined host clock, and SHALL report the source and synced honestly in SLC-CP get_clock_status. Only a device with no time source whatsoever counts seconds from start_rx, and it SHALL say so on the wire with TSI=11b. A client SHALL treat a TSI=11b timestamp as relative and SHALL NOT display it as a time of day.

This aligns with SDR-SLC-CP-1.0 Release 1.3 §20 and replaces the Release 1.1 rule, under which a free-running device counted from stream start under TSI=01b — a UTC mode code carrying a non-UTC value, which no VITA-49 tool could tell apart from the real thing.

6.4.2 Timestamp Fractional (Picoseconds)

A 64-bit unsigned integer split across two 32-bit words: high word at offset +20, low word at offset +24. Contains the sub-second offset in picoseconds from the start of the current second.

```
picos = (sample_count mod fs_hz) * 10^12 / fs_hz

sample_count = number of IQ pairs output since the last start_rx call
fs_hz        = nominal sample rate in samples per second

CRITICAL: Use integer arithmetic only (uint64_t in C). Floating-point
gives rounding errors that accumulate over time.

Example at 2,048,000 SPS with 360 pairs per packet:
Packet 0: sample_count = 0   picos = 0
Packet 1: sample_count = 360 picos = 360 * 10^12 / 2048000 = 175,781,250
Packet 2: sample_count = 720 picos = 720 * 10^12 / 2048000 = 351,562,500

sample_count resets to 0 on each start_rx call.
The integer seconds field increments by 1 each time
(sample_count mod fs_hz) wraps to a smaller value.
```

The fractional timestamp SHALL be derived from the nominal sample rate accepted by set_sample_rate, not from the achieved rate reported as actual_sample_rate_sps. The nominal rate divides exactly; the achieved rate does not, and using it slips the client's timeline by one sample per second. On a device whose achieved rate differs, the integer second will drift slowly against true UTC, and that drift is exactly the information actual_sample_rate_sps carries.

6.5 I/Q Payload Encodings

The I/Q payload immediately follows the 28-byte header. Samples are interleaved: I0 Q0 I1 Q1 ... The encoding is negotiated via SLC-CP before streaming begins and remains constant for the entire stream lifetime. All encodings use big-endian byte order.

Label	Encoding	Bits	Bytes/pair	Notes
u8	Unsigned offset binary	8	2	Native on RTL-SDR. Zero signal at zero_point, nominally 128. Lowest bandwidth option.
int16	Two's complement	16	4	Native on most 16-bit ADCs and on the bladeRF after a left shift of its 12-bit samples (a scale, not a requantisation, so native:true is still honest).
int24	Two's complement	24	8	24-bit value left-justified in a 32-bit container; lower 8 bits zero.
float32	IEEE 754 single	32	8	±1.0 corresponds to full-scale ADC range.

The payload SHALL be a whole number of 32-bit words: samples_per_packet × bytes_per_pair SHALL be a multiple of 4. For u8 this means an even number of pairs; for the other encodings any count qualifies. The Packet Size field cannot express a half word, and VITA-49.0 does not permit one.

Zero point for unsigned encodings. For encoding: "unsigned" the sample value that represents zero signal is zero_point, declared on the format entry in iq_formats and echoed in the open_stream response. When absent it defaults to 2^(bits_per_component - 1): 128 for u8. A receiving client SHALL subtract zero_point from each component before scaling to full scale, and a transmitting client SHALL add it before packing. zero_point is a JSON number and need not be an integer. It SHALL NOT appear on a twos_complement or float32 entry, and a client SHALL ignore it there. It is a static calibration of the converter's midpoint, not DC removal: a client that also removes

DC does so on top of it, and `full_scale` in the power formula is unaffected — the amplitude range stays 127 counts either side of the declared zero.

7. VITA-A/1.0-CTX Context Packet Profile [RX-T2]

Context packets make the VITA-SLC stream self-describing. Without them, the client must obtain signal type and calibration information from the SLC-CP control channel and keep it in sync manually. With Context packets, the UDP stream itself carries this information in-band, adjacent to the data it describes.

Two pieces of information are carried:

- Signal type and encoding — whether the payload contains complex I/Q or a real-valued signal from one ADC branch (RTL-SDR direct sampling mode), and how the samples are encoded. The client's DSP pipeline must know this before it can demodulate correctly.
- Reference Level — the signal power at the antenna port that corresponds to full-scale samples at the current gain setting. Changes with every gain adjustment. Enables calibrated power display.

WARNING

Normal tuner mode and direct sampling mode are not minor variants. A DSP pipeline that applies complex quadrature demodulation to a real direct-sampling signal produces completely incorrect output. The client must know the mode before processing the first sample.

A Tier 1 device does not send Context packets. The client assumes complex I/Q mode, takes the encoding from the `open_stream` echo, and uses the Reference Level from SLC-CP capabilities (if present) or displays power as uncalibrated. A Tier 2 device sends a Context packet before the first IF Data packet after `start_rx` and after any gain or mode change.

7.1 Context Packet Layout

Figure 7 -- Complete VITA-A/1.0-CTX Context Packet Layout

Offset	Size	Field
+0	4 bytes	VRT Header
		[31:28]=0100b Packet Type: IF Context + Stream ID
		[27]=0 No Class Identifier
		[26]=0 No Trailer
		[25:24]=00b Reserved (always 0)
		[23:22]=00b TSI: No Timestamp
		[21:20]=00b TSF: No Timestamp
		[19:16]=0..F Packet Count (separate from data)
		[15:0]=0x0006 Packet Size: 6 words (fixed)
+4	4 bytes	Stream Identifier (same value as IF Data stream)
+8	4 bytes	CIF0 = 0x01008000, or 0x81008000 when a field changed
		bit 31 = Context Field Change Indicator
		bit 24 = Reference Level present
		bit 15 = Data Packet Payload Format present
		all other bits = 0
+12	4 bytes	Reference Level
		[31:16] = 0x0000 (reserved)
		[15:0] = signed Q9.7 dBm (1/128 dBm per LSB)
+16	4 bytes	Payload Format Word 0 (per negotiated encoding, table 7.5)
+20	4 bytes	Payload Format Word 1: 0x00000000 (always)

Total packet: 6 words x 4 bytes = 24 bytes fixed size.

VRT Header constant for Context: 0x40000006 | (count <= 16)

7.2 VRT Header for Context Packets

The Context packet VRT Header differs from the IF Data header in four ways: the Packet Type is 0x4 (IF Context) instead of 0x1, Class ID Present is 0, TSI is 00b and TSF is 00b. The Packet Size is always 6.

Bits	Field	Value	Notes
[31:28]	Packet Type	0100b	IF Context Packet with Stream ID.
[27]	Class ID Present	0	No Class Identifier in Context packets.
[26]	Trailer Present	0	No trailer. Always 0.

Bits	Field	Value	Notes
[25:24]	Reserved	00b	Always 0. (VITA-49.0 uses bit 24 of a context header as TSM; SDR-SLC context packets carry no timestamp, so it is 0.)
[23:22]	TSI	00b	No Timestamp. Context packets describe the stream state, not a sample time.
[21:20]	TSF	00b	No Timestamp Fractional.
[19:16]	Packet Count	0..15	Modulo-16 counter for Context packets. Maintained independently from the IF Data packet count. A client SHALL NOT use this to infer data packet loss.
[15:0]	Packet Size	6	1 (header) + 1 (stream ID) + 1 (CIF0) + 1 (Reference Level) + 2 (Payload Format) = 6 words.

7.3 Context Indicator Field (CIF0)

CIF0 is a 32-bit bitmask at offset +8. Each set bit indicates that a corresponding context field is present in the packet body, in descending bit order. The bit assignments are VITA-49.0 §7.1.5.1; the ones SDR-SLC uses are:

CIF0 = 0x01008000 (or 0x81008000 when a field has changed)

bit 31 (0x80000000) Context Field Change Indicator: set when the Reference Level or the Payload Format differs from the previous Context packet on this stream. Set on the first packet.
bit 24 (0x01000000) Reference Level field present at offset +12
bit 15 (0x00008000) Data Packet Payload Format field present at offset +16
all other bits = 0

Field ordering within the packet body follows CIF0 bit order, most significant set bit first. Bit 24 > bit 15, so Reference Level comes before Payload Format.

For reference, the neighbouring VITA-49.0 CIF0 assignments a future revision might use: bit 27 RF Reference Frequency, bit 23 Gain, bit 22 Over-range Count, bit 21 Sample Rate, bit 29 Bandwidth. A receiver SHALL check bits 24 and 15 and SHALL ignore the packet if either is clear; it SHALL NOT assume any other bit is clear (§14).

NOTE

Release 1.1 placed these fields in bits 7 and 1. VITA-49.0 reserves both; a standard dissector shows a Release 1.1 packet as carrying no fields at all. If a client checks CIF0 against 0x82, it is decoding Release 1.1.

7.4 Reference Level Field

A 32-bit word at offset +12, encoded per VITA-49.0 §7.1.5.9. Bits 31:16 are reserved and SHALL be zero. Bits 15:0 are a signed 16-bit two's-complement value with the radix point between bits 6 and 7 — a Q9.7 fixed-point number of dBm, 1/128 dBm per LSB, range -256.0 to +255.99 dBm.

Encoding: raw = int16(round(dBm * 128))
word = 0x0000_0000 | (uint16) raw
Decoding: dBm = int16(word & 0xFFFF) / 128.0

-30.0 dBm:	-30.0 * 128 = -3840	= 0xF100	-> word 0x0000F100
-40.0 dBm:	-5120	= 0xEC00	-> word 0x0000EC00
-55.0 dBm:	-7040	= 0xE480	-> word 0x0000E480
-10.5 dBm:	-1344	= 0xFAC0	-> word 0x0000FAC0
-25.75 dBm:	-3296	= 0xF320	-> word 0x0000F320

C implementation:
int16_t rl_q7 = (int16_t)lroundf(ref_level_dbm * 128.0f);
uint32_t word = (uint32_t)(uint16_t)rl_q7;
write_be32(buf+12, word);

TIP

If your device does not have factory calibration data, use a hardcoded nominal Reference Level such as -30.0 dBm at the preferred gain setting, and publish reference_level_accuracy_db in capabilities. Track gain changes with new_RL = old_RL - gain_change_db.

7.5 Payload Format Field

Two 32-bit words at offsets +16 and +20 form the VITA-49.0 §7.1.5.18 Data Packet Payload Format field. Word 0 describes the signal type and encoding; word 1 carries the repeat count and vector size and is always zero in SDR-SLC profiles (both encode as "1").

Word 0 bit layout (VITA-49.0 7.1.5.18):		
[31]	Packing Method	0 = processing-efficient (always, in SDR-SLC)
[30:29]	Real/Complex Type	00 = real, 01 = complex Cartesian
[28:24]	Data Item Format	00000 = signed fixed-point
		10000 = unsigned fixed-point
		01110 = IEEE-754 single precision
[23]	Sample-Component Repeat	0
[22:20]	Event-Tag Size	000
[19:16]	Channel-Tag Size	0000
[15:12]	Reserved	0000
[11:6]	Item Packing Field Size	(field size in bits) - 1
[5:0]	Data Item Size	(item size in bits) - 1
Word 1:		
[31:16]	Repeat Count - 1	0
[15:0]	Vector Size - 1	0

The size fields encode size - 1, so a 16-bit item is 001111b. Getting this off by one is the classic error; the table below is the whole set of constants an SDR-SLC device needs.

Negotiated encoding	Complex I/Q (word 0)	Real (word 0)	Derivation
int16	0x200003CF	0x000003CF	signed, item 16 (0x0F), packing 16 (0x0F)
u8	0x300001C7	0x100001C7	unsigned, item 8 (0x07), packing 8 (0x07)
int24	0x200007D7	0x000007D7	signed, item 24 (0x17), packing field 32 (0x1F)
float32	0x2E0007DF	0x0E0007DF	IEEE-754 single, item 32 (0x1F), packing 32 (0x1F)

A device SHALL send the word 0 for the encoding negotiated in `open_stream` and the current real/complex mode. A client SHALL determine real versus complex from bits [30:29] and SHOULD verify that bits [5:0] agree with the negotiated encoding, logging a mismatch. A client SHALL NOT match the whole word against a constant, so that a future revision may set bits it does not use.

NOTE	Release 1.1 used 0x0F8700E0 for every encoding. Under the VITA-49.0 layout that word decodes as a real signal with an undefined item format, and its annotation in that release did not match the standard's field positions. It is not accepted by Release 1.2 receivers.
-------------	--

7.6 Gain Tracking: When to Send Context Packets

A Context packet must be sent before the first IF Data packet whenever the Reference Level or the Payload Format changes. The three triggers are:

- After `start_rx`: always send a Context packet first, with bit 31 set.
- After a gain control changes: when `rf_gain`, `lna_gain`, `if_gain`, or any other control that affects signal path gain is changed via SLC-CP `set_control` or by AGC, update the Reference Level (`new_RL = old_RL - gain_change_dB`) and send a Context packet.
- After `direct_sampling` changes: the Payload Format changes (complex ↔ real); send a Context packet.

A device MAY also send an unchanged Context packet periodically (bit 31 clear) so that a late-joining observer (SDR-SLC-CP-1.0 §17.4) learns the stream state without waiting for a change. Once per second is ample.

7.7 Receiver Behaviour (Client Rules)

- A client that has NOT implemented Context packet parsing SHALL silently discard any UDP packet whose VRT Header Packet Type field is 0100b (0x4). Do not treat it as a data packet loss.
- A client that HAS implemented Context packet parsing SHOULD inspect the Context packet before processing any IF Data packets for a new stream.
- A client SHALL NOT use the Context packet Packet Count field to infer IF Data packet loss. The two counters are independent.
- A client SHOULD update its DSP pipeline and power display immediately upon receiving a Context packet.
- The Context packet is authoritative. If the Context packet says the mode is real but the SLC-CP signal_type field in the open_stream response says complex_iq, the client SHALL use the Context packet value. The Context packet reflects the current hardware state; the open_stream response reflects the state at the time the stream was opened.
- If no Context packet has been received for a stream, the client MAY assume complex I/Q mode, take the encoding from the open_stream echo, and use the Reference Level from SLC-CP get_capabilities or get_status. This is the Tier 1 client fallback.

7.8 Complete Annotated Packet Examples

7.8.1 IF Data Packet: 360 pairs, u8 encoding, Stream ID=1, Packet Count=0

Clock: 2026-09-14T09:12:01Z (epoch 1789377121), sample 0.

Offset	Bytes	Field and value
+00	18 00 00 BB	VRT Header: 0x180000BB [31:28]=0001b IF Data + Stream ID [27]=1 Class ID [26]=0 no trailer [25:24]=00b [23:22]=01b TSI UTC [21:20]=10b TSF picoseconds [19:16]=0000b count 0 [15:0]=0x00BB 187 words (7 header + 180 payload; 360 u8 pairs / 2 per word)
+04	00 00 00 01	Stream Identifier: 1
+08	00 40 28 14	Class ID Word 0: reserved=0x00, OUI=0x402814
+12	00 00 00 01	Class ID Word 1: ICC=0x0000, PCC=0x0001
+16	6A A7 BA 61	Timestamp Integer: 0x6AA7BA61 = 1789377121
+20	00 00 00 00	Timestamp Fractional High: 0
+24	00 00 00 00	Timestamp Fractional Low: 0 -> 0 ps (first packet)
+28	80 80 80 80	Payload: I0=0x80 Q0=0x80 I1=0x80 Q1=0x80 (zero signal)
...		continuing for 720 bytes = 360 IQ pairs

Total UDP payload: 28 + 720 = 748 bytes. IP datagram: 776 bytes.

7.8.2 Context Packet: u8 complex, rf_gain=20 dB, RL=-30.0 dBm, Stream ID=1

+00	40 00 00 06	VRT Header: 0x40000006 [31:28]=0100b IF Context + Stream ID [27]=0 no CID [23:22]=00b [21:20]=00b no timestamp [19:16]=0000b count 0 [15:0]=0x0006 6 words
+04	00 00 00 01	Stream Identifier: 1 (same as IF Data stream)
+08	81 00 80 00	CIF0: 0x81008000 bit 31 = changed (first packet) bit 24 = Reference Level present (+12) bit 15 = Payload Format present (+16)
+12	00 00 F1 00	Reference Level: 0x0000F100 low 16 = 0xF100 = -3840 (int16) / 128 = -30.0 dBm
+16	30 00 01 C7	Payload Format Word 0: 0x300001C7 [30:29]=01b complex Cartesian [28:24]=10000b unsigned fixed-point [11:6]=000111b packing 8 bits [5:0]=000111b item 8 bits
+20	00 00 00 00	Payload Format Word 1: repeat 1, vector 1

Total UDP payload: 24 bytes. IP datagram: 52 bytes.

This packet MUST be sent before the first IF Data packet after start_rx.

7.8.3 Context Packet: After Gain Increase to 30 dB (RL now -40.0 dBm)

```
+00    40 10 00 06  VRT Header: Packet Count = 1
+04    00 00 00 01  Stream ID: 1
+08    81 00 80 00  CIF0: changed
+12    00 00 EC 00  Reference Level: 0xEC00 = -5120 / 128 = -40.0 dBm
+16    30 00 01 C7  Payload Format W0: unchanged, u8 complex
+20    00 00 00 00  Payload Format W1: 0
```

7.8.4 Context Packet: Direct Sampling Mode Enabled (real signal)

```
+00    40 20 00 06  VRT Header: Packet Count = 2
+04    00 00 00 01  Stream ID: 1
+08    81 00 80 00  CIF0: changed
+12    00 00 EE 80  Reference Level: 0xEE80 = -4480 / 128 = -35.0 dBm
                    (direct sampling path has a different gain budget)
+16    10 00 01 C7  Payload Format W0: 0x100001C7
                    [30:29]=00b real (identifies direct sampling mode)
+20    00 00 00 00  Payload Format W1: 0
```

7.8.5 Context Packet: int16 complex stream (bladeRF), RL=-55.0 dBm

```
+00    40 00 00 06  VRT Header: Packet Count = 0
+04    00 00 00 0C  Stream ID: 12
+08    81 00 80 00  CIF0: changed
+12    00 00 E4 80  Reference Level: 0xE480 = -7040 / 128 = -55.0 dBm
+16    20 00 03 CF  Payload Format W0: 0x200003CF
                    [30:29]=01b complex [28:24]=00000b signed
                    [11:6]=001111b packing 16 [5:0]=001111b item 16
+20    00 00 00 00  Payload Format W1: 0
```

7A. VITA-A/1.0-TX Transmit IF Data Profile [TX-T1 accept • TX-T2 send]

The transmit profile carries I/Q from the client to the device. It is identical to VITA-A/1.0 except where stated, and each difference has a reason. A tx Tier 1 device accepts plain VITA-A/1.0 packets on its `listen_port` as continuous transmission and need not implement anything in this section beyond §7A.6. A tx Tier 2 device accepts this profile and places bursts on the sample it names.

	VITA-A/1.0 (receive)	VITA-A/1.0-TX (transmit)
Direction	device → client	client → device
UDP port	device <code>source_port</code> (4621)	device <code>listen_port</code> (4622)
PCC	0x0001	0x0002
TSI	01b UTC seconds (11b relative)	00b none
TSF	10b picoseconds	01b sample count
Trailer	absent	present; carries burst delimiters
Header size	28 bytes (7 words)	24 bytes (6 words)
<code>samples_per_packet</code>	fixed per stream	advisory; device reads the size field

7A.1 Packet Layout

Offset	Size	Field
+0	4 bytes	VRT Header
		[31:28]=0001b IF Data Packet with Stream ID
		[27]=1 Class ID present
		[26]=1 Trailer present
		[25:24]=00b Reserved
		[23:22]=00b TSI: no integer timestamp
		[21:20]=01b TSF: sample count

		[19:16]	Packet Count, modulo 16
		[15:0]	Packet Size in 32-bit words
+4	4 bytes	Stream Identifier: vita.stream_id from open_stream (tx)	
+8	4 bytes	Class ID Word 0: 0x00402814	
+12	4 bytes	Class ID Word 1: ICC 0x0000, PCC 0x0002 -> 0x00000002	
+16	8 bytes	Transmit timestamp: 64-bit TX sample count, high word first. 0 = transmit now.	
+24	N x 4 B	I/Q payload, N pairs, interleaved, negotiated encoding	
last	4 bytes	Trailer: burst delimiters	

Packet Size in words = 6 + N + 1
Header constant, excluding count and size: 0x1C100000

7A.2 The timestamp is a sample count, not a time

TSF=01b (Sample Count) rather than the receive profile's picoseconds, and TSI=00b because there is no integer part. The value is the device's own transmit sample counter, the same counter that SLC-CP get_timestamps returns as tx_sample_count. It is not a wall clock and does not need one.

This is the load-bearing decision in the profile. A burst has to land on an exact sample — DMR wants 27.5 ms of energy inside a 30 ms slot — and the only clock that is exact is the device's. Routing the schedule through UTC would mean converting via the host clock, which is good to milliseconds: three orders of magnitude short of the requirement.

timestamp = 0 means transmit as soon as possible, continuously. Use it for FM, SSB, and anything else without slot structure.

7A.3 Obtaining the mapping

SLC-CP get_timestamps (SDR-SLC-CP-1.0 §15.8) returns rx_sample_count, tx_sample_count, both rates, shares_clock, capture_spread_ns and rx_stream_epoch_sample. A client converts the receive sample index of an observed event into a transmit schedule:

```
rx_abs    = rx_stream_epoch_sample + elapsed_rx_samples(from the VITA timestamp)
tx_sched = tx_sample_count + (rx_abs - rx_sample_count) * tx_rate / rx_rate
```

valid when shares_clock is true. capture_spread_ns bounds the error of the read; check it. If the device emits rx_epoch_changed, re-read before the next burst.

7A.4 Burst delimiting

VITA-49.0 defines no start- or end-of-burst indicator, so the two user-defined trailer indicator bits carry it, with their enable bits set as VITA-49.0 §7 requires for any indicator to be honoured.

```
Trailer word:
bit 23 enable for bit 11 } set both to assert
bit 11 BURST_START      }
bit 22 enable for bit 10 } set both to assert
bit 10 BURST_END        }
all other bits 0

burst start: 0x00800800
burst end:   0x00400400
both:        0x00C00C00    (a burst of one packet)
neither:     0x00000000    (mid-burst continuation)
```

Semantics:

- BURST_START with a non-zero timestamp schedules the burst to begin on that exact TX sample. The PA is gated off until then. The packet SHALL arrive at least min_burst_lead_us before the scheduled sample.
- BURST_START with timestamp 0 begins immediately.
- BURST_END closes the burst after this packet's payload. The PA is gated off afterwards, so nothing is radiated between bursts.
- Packets with neither continue the open burst. Their timestamp is ignored.

- A packet arriving while no burst is open implicitly starts one. A client that never sets the trailer bits therefore gets continuous transmission, which is the correct behaviour for FM and SSB.

A burst whose timestamp has already passed when it reaches the device SHALL be dropped, not transmitted late: energy in a slot that has moved on is worse than no energy. The device emits `tx_burst_late` with `samples_late`. A client SHALL treat that event as its scheduling being too tight, not as a transport error.

7A.5 Payload

Identical to the receive profile: interleaved I/Q pairs, most significant byte first, in the encoding negotiated through `open_stream`. Full scale is ± 32767 for int16; the device scales to the DAC's native range itself.

`samples_per_packet` in `capabilities.tx.transports[]` is advisory for this direction. The device SHALL read the pair count from each packet's Packet Size field, and SHALL accept any count that fits the MTU. The §6.5 word-alignment rule applies.

7A.6 Accepting the receive profile on the transmit port [TX-T1]

A plain VITA-A/1.0 packet — PCC 0x0001, TSI=01b, TSF=10b, no trailer — arriving on the transmit port SHALL be accepted and treated as continuous transmission starting now. There is no ambiguity about direction on that port, and it lets a client with only a receive-side encoder transmit voice today. This is the whole of tx Tier 1.

Its timestamp SHALL be discarded, not reinterpreted. A receive-profile timestamp says when samples were captured; treating that as a transmit schedule would be meaningless and would appear to work. Scheduled bursts require this profile.

7A.7 Data timeout

SDR-SLC-CP-1.0 §15.7 requires the device to stop transmitting when no data has arrived for `tx_data_timeout_ms`. The timeout counts from the last packet received. A client pre-loading a burst may go quiet until the next one as long as the gap is shorter than the timeout; a client transmitting continuously SHALL keep feeding at the sample rate.

7A.8 Worked example

One 256-pair burst-start packet scheduled at TX sample 91,300,000, stream ID 2, packet count 0:

+00	1C 10 01 07	header: type 1, CID, trailer, TSI=00, TSF=01, count 0, size 263 words = 1052 bytes
+04	00 00 00 02	stream id 2
+08	00 40 28 14	OUI 40-28-14
+12	00 00 00 02	ICC 0x0000, PCC 0x0002
+16	00 00 00 00	timestamp high
+20	05 71 20 A0	timestamp low = 91,300,000
+24	...	256 pairs, int16 big-endian, 1024 bytes
+1048	00 80 08 00	trailer: BURST_START

7A.9 Client conformance checklist

- Header constant 0x1C100000, OR'd with count and size in words.
- Packet Size counts every word including the trailer.
- Class ID 0x00402814 / 0x00000002.
- Timestamp is a TX sample count from `get_timestamps`, not a wall clock.
- Enable bits set alongside any trailer indicator.
- BURST_END on the last packet of a burst, or the PA stays keyed until the data timeout.
- Payload big-endian, interleaved, full scale ± 32767 .
- Keep feeding an open burst: a gap longer than `tx_data_timeout_ms` disarms the transmitter and requires a new `start_tx`.
- Re-read `get_timestamps` after any `rx_epoch_changed` event.

8. I/Q Format Negotiation

The I/Q encoding used in IF Data packets is negotiated via SLC-CP before streaming begins. The sequence is:

1. Client calls `get_capabilities` and reads `capabilities.rx.iq_formats[]` (or `capabilities.tx.iq_formats[]`). Each entry has an `encoding`, `bits_per_component`, `container_bits`, `packing`, `byte_order`, `native`, `supported`, and `label` field.
2. Client selects a format entry where `supported` is true. The entry with `native:true` requires no hardware conversion and minimises latency.
3. Client includes the selected format object in the `open_stream` request as `params.vita.format`.
4. Device confirms the format in the `open_stream` response as `result.vita.format`, along with `result.vita.samples_per_packet`. The client SHALL use the echoed format object and pair count, not the ones it sent, to decode packets — the device may substitute a compatible variant, and for int24/float32 reduces the pair count to fit the MTU (Appendix A).
5. The negotiated format applies for the entire stream. If the client wants a different format, it must close the stream and open a new one.

NOTE	If the client omits the format field from the <code>open_stream</code> request, the device uses the format with <code>native:true</code> as the default. At Tier 1, a device may ignore the format request entirely and always use its native format; the echo tells the client so.
-------------	---

Field	Type	Description
<code>encoding</code>	string	"twos_complement", "unsigned", or "float32".
<code>bits_per_component</code>	integer	Significant bits per I or Q component (8, 16, 24, 32).
<code>container_bits</code>	integer	Size of the storage container in bits. Equals <code>bits_per_component</code> for int16 and u8; 32 for int24.
<code>packing</code>	string	Always "interleaved" (I Q I Q ...).
<code>byte_order</code>	string	Always "big_endian".
<code>native</code>	boolean	true if this format is produced or consumed directly by the converter.
<code>supported</code>	boolean	true if the device can currently use this format. A device may list formats with <code>supported:false</code> to document planned support.
<code>label</code>	string	Short human-readable identifier: u8, int16, int24, float32.
<code>zero_point</code>	number	Unsigned encodings only, optional. Sample value that represents zero signal (§6.5). Default $2^{(\text{bits_per_component} - 1)}$. Echoed in the <code>open_stream</code> response.

9. Stream Identification and Multiplexing

Every UDP packet — IF Data, Context or transmit — carries a 32-bit Stream ID in the second word of the packet. The Stream ID links packets to the stream session that was established via SLC-CP `open_stream`.

- The device assigns the numeric Stream ID and returns it in the `open_stream` response as `vita.stream_id`, for both receive and transmit streams.
- The Stream ID SHALL NOT be reused within a TCP control connection session.
- The Stream ID is shared between IF Data packets and Context packets for the same stream.
- When the client has multiple streams open simultaneously, it demultiplexes received packets first by Stream ID, then by Packet Type (VRT Header bits 31:28): 0x1 = IF Data, 0x4 = Context.
- The Packet Count field is maintained separately for IF Data packets and Context packets within each stream, and separately per direction.

TIP	For a single-stream device, the numeric Stream ID can start at 1 and count up per <code>open_stream</code> . The SLC-CP <code>stream_id</code> string ("rx-1") is a different identifier and may repeat; both are returned by <code>open_stream</code> , one as <code>stream_id</code> , the other as <code>vita.stream_id</code> .
------------	---

10. Clock and Timestamp Model

Every receive IF Data packet carries a timestamp. Its meaning is fixed by the TSI bits (§6.4.1) and its quality by the device's clock source, which the client queries with SLC-CP `get_clock_status`.

Source	Accuracy	Notes
GPS-locked PTPv2 grandmaster	< 1 μ s	Best. UTC-traceable. TSI=01b, synced:true.
Non-GPS PTPv2 grandmaster	< 100 μ s	Good. TSI=01b, synced:true.
GPS receiver on device	< 1 μ s	Good. TSI=01b, synced:true.
Manual (SLC-CP <code>set_clock</code>)	Client accuracy	TSI=01b, synced:false.
Host clock (free_running)	Host NTP accuracy, ms	TSI=01b, synced:false. Both reference daemons.
No time source	Relative only	TSI=11b. Seconds since <code>start_rx</code> .

NOTE	Context packets carry no timestamp at all (TSI=TSF=00b). Transmit packets carry a sample count (TSF=01b), which is not a time. The only wall-clock timestamp in the suite is on receive IF Data packets.
-------------	--

Clients SHOULD call `get_clock_status` at startup and warn the operator if `syncd` is false. Timestamps from such a device are only as good as its host clock and should not be used for absolute time measurements.

10.1 PTP Integration

If your device uses PTPv2 (IEEE 1588-2008), a grandmaster on the LAN provides TAI time via SYNC and FOLLOW_UP messages. The device converts TAI to UTC by subtracting the UTC offset carried in the PTPv2 ANNOUNCE message (37 seconds as of 2026). The VITA-49 Timestamp Integer field carries UTC seconds since the Unix epoch.

TIP	PTP is optional. A host clock is acceptable for Tier 1 and Tier 2. The <code>ptpd</code> or <code>linuxptp</code> software stacks are available for Linux; for bare-metal STM32, small PTP slave implementations exist.
------------	---

11. Multi-Stream Operation

Most SDR-SLC implementations handle one active RX stream at a time. Hardware with multiple independent receiver chains declares them as channels (SDR-SLC-CP-1.0 §12.3) and can support concurrent streams with distinct Stream IDs, sample rates, and UDP destinations.

- Each concurrent stream has its own Stream ID, its own Packet Count counter, its own Context packet sequence, and its own UDP destination.
- The device SHALL NOT send packets for a stream that has been stopped or closed.
- Single-chain hardware enforces the one-active-stream-per-direction rule in the SLC-CP `open_stream` logic by returning `stream_already_open`.
- There is no capacity field. Capacity is a runtime condition, reported by `busy` (SDR-SLC-CP-1.0 §17.5).

12. Conformance Requirements

12.1 IF Data Packet Sender [RX-T1]

- Transmit VITA-49 IF Data packets with Packet Type 0x1 (0001b) and Stream ID.
- Set Class ID Present (bit 27) = 1 in all IF Data VRT Headers.
- Set Trailer Present (bit 26) = 0.
- Set TSI = 01b when any time source exists; 11b only when none does.
- Set TSF = 10b (picoseconds).
- Use OUI = 0x402814, ICC = 0x0000, PCC = 0x0001.
- Compute Timestamp Fractional from the nominal sample rate using integer arithmetic only.

- Reset `sample_count` to zero on each `start_rx` call.
- Increment Packet Count modulo 16 for each IF Data packet. Maintain independently per stream.
- Honour the negotiated I/Q format for the entire stream lifetime.
- Keep the payload a whole number of 32-bit words; fit each IP datagram within 1500 bytes.
- Use big-endian byte order for all fields and all payload encodings.
- Assign a unique numeric Stream ID per stream. Do not reuse within a TCP session.
- Stop sending packets immediately on TCP control connection loss.

12.2 Context Packet Sender [RX-T2]

- All RX-T1 requirements.
- Emit a VITA-A/1.0-CTX Context packet before the first IF Data packet after `start_rx`, after any `direct_sampling` mode change, and after any gain control change including AGC.
- Set Context packet VRT Header: Type=0x4, CID=0, TSI=00b, TSF=00b, Size=6.
- Set CIFO bits 24 and 15; set bit 31 when a field changed; all other bits 0.
- Encode Reference Level as Q9.7 in bits 15:0 with bits 31:16 zero.
- Send the Payload Format word 0 for the negotiated encoding and current mode (table §7.5); word 1 zero.
- Maintain a separate Packet Count for Context packets.
- Update Reference Level correctly on each gain change: $\text{new_RL} = \text{old_RL} - \text{gain_change_dB}$.
- Rate-limit Context packet emission to no more than one per 100 ms during AGC operation.

12.3 Transmit Receiver [TX-T1] and Burst Receiver [TX-T2]

- [TX-T1] Accept plain VITA-A/1.0 packets on `listen_port` as continuous transmission; discard their timestamps.
- [TX-T1] Validate OUI and PCC; discard packets with other values.
- [TX-T1] Read the pair count from the Packet Size field; accept any count that fits the MTU.
- [TX-T1] Stop transmitting when no packet has arrived for `tx_data_timeout_ms`.
- [TX-T2] Accept VITA-A/1.0-TX packets: PCC 0x0002, TSI=00b, TSF=01b, trailer present.
- [TX-T2] Honour BURST_START with a timestamp by starting on that sample; gate the PA off outside bursts.
- [TX-T2] Drop a late burst and emit `tx_burst_late`.
- [TX-T2] Implement SLC-CP `get_timestamps`.

12.4 Client Requirements

- Decode IF Data payload using the echoed format and pair count from the SLC-CP `open_stream` response.
- Demultiplex received UDP packets first by Stream ID, then by Packet Type.
- Validate OUI and PCC; discard packets with other values.
- Silently discard any packet with VRT Packet Type 0x4 if Context packet parsing is not implemented.
- NOT infer IF Data packet loss from Context packet Packet Count.
- Treat Stream ID as opaque.
- Treat a TSI=11b timestamp as relative.
- Request only I/Q formats where supported: `true`.
- For an unsigned encoding: subtract the echoed `zero_point` (default $2^{(\text{bits}-1)}$) from each component before scaling, and add it before packing on transmit; ignore `zero_point` on any other encoding.
- When parsing Context packets: check CIFO bits 24 and 15; decode Reference Level as Q9.7; take real/complex from Payload Format bits [30:29]; never match the whole Payload Format word against a constant.
- SHOULD treat the Context packet Reference Level and signal type as authoritative over SLC-CP fields.
- SHOULD validate the UDP source address of received packets against `vita.source.ip` from the `open_stream` response, and discard others.
- When transmitting: follow the §7A.9 checklist.

13. Security Considerations

VITA-SLC is designed for trusted local area network operation. It provides no authentication, encryption, or integrity verification. Any host on the LAN that can send UDP packets to a listening port can inject fabricated packets.

The practical risks for a home station are: a spoofed IF Data packet injects false signal data into the waterfall; a spoofed Context packet with a fabricated Reference Level causes the client to display incorrect power readings; a spoofed Context packet changing the Payload Format could cause the DSP pipeline to misinterpret subsequent real data; and, on the transmit port, a spoofed packet radiates whatever it contains under the licence holder's call sign.

Mitigation: a client SHOULD validate the source IP address of received UDP packets against the device address returned by SLC-CP `open_stream`, and a device SHOULD accept transmit data only from the `vita.source` the client declared. Both are a one-line filter in the receive path. SLC-CP §26 authentication protects the control plane that sets these up; it does not protect the UDP path, which is why the source filter matters on the transmit port in particular.

WARNING

On an unmanaged home LAN with no other users, the receive-side risks are negligible. The transmit-side filter should always be on.

14. Forward Compatibility

The SDR-SLC profiles are designed to be extended without breaking existing receivers:

- Reserved bits in the VRT Header (bits [25] and [24]) SHALL be set to 0 by transmitters and SHALL be ignored by receivers.
- CIFO bits other than 31, 24 and 15 SHALL be set to 0 by this release's senders and SHALL be tolerated by receivers. When a receiver encounters a set CIFO bit it does not know, it SHALL skip the corresponding field using the field sizes VITA-49.0 defines, or discard the packet. A future revision MAY add Sample Rate (bit 21), RF Reference Frequency (bit 27) or Gain (bit 23) this way; a receiver that ignores them keeps working.
- Payload Format bits a receiver does not use SHALL be ignored; a receiver SHALL NOT compare the whole word against a constant.
- New transport format labels (beyond u8, int16, int24, float32) MAY be added with `supported:false` until implemented.
- A future VITA-A/2.0 profile MAY use a different PCC value to allow coexistence on the same network. Receivers SHALL ignore packets whose PCC they do not know.

NOTE

When processing packets of an unknown Packet Type (other than 0x1 or 0x4), receivers SHOULD discard the packet and log an anomaly rather than attempting to decode the payload.

15. References

15.1 Normative References

Reference	Title	Relevance
ANSI/VITA 49.0	VITA Radio Transport Standard (VRT)	Base VRT packet format. §6.1 (VRT header), §7.1.5.1 (CIFO), §7.1.5.9 (Reference Level), §7.1.5.18 (Data Packet Payload Format), §7 (trailer).
RFC 768	User Datagram Protocol	UDP transport for all VITA-SLC packets.
IEEE 1588-2008	Precision Time Protocol v2	PTPv2 clock synchronisation for UTC timestamps.
IEEE 802 / OUI	IEEE OUI Registry	IXChange BV OUI 40-28-14 (0x402814).
SDR-SLC-CP-1.0 Release 1.3	SLC-CP Control Plane Specification	Stream lifecycle, format negotiation, <code>signal_type</code> , <code>get_timestamps</code> , clock status, transmit safety.
SDR-SLC-PS-1.0	SDR-SLC Protocol Suite Architecture	Overall system architecture and design rationale.
RFC 2119	Key Words for Use in RFCs	Normative keyword definitions.

15.2 Informative References

Reference	Title	Notes
ANSI/VITA 49.2-2017	VITA Radio Transport Standard v2	Superset of 49.0; same CIFO, Reference Level and Payload Format definitions. May form the basis of a future VITA-A/2.0 profile.
SDR-SLC-DISC-1.0	SLC-DISC Service Discovery	How clients discover device IP and port.
sdr-slc-rtld	Reference implementation, RTL-SDR bridge	u8, Context packets, host-clock timestamps.
sdr-slc-bladerfd	Reference implementation, bladeRF 2.0 micro	int16, Context packets, VITA-A/1.0-TX with scheduled bursts.
lwIP	Lightweight IP Stack	Recommended TCP/IP stack for STM32H7 implementations.
cJSON	Ultralight JSON Parser	Recommended JSON library for embedded implementations.

Appendix A. Packet Size Reference Table

IP datagram sizes for all supported encoding depths and pair counts. IP header = 20 bytes, UDP header = 8 bytes, VITA-A/1.0 header = 28 bytes.

Encoding	Bytes/pair	360 pairs (default)	256 pairs	180 pairs	MTU safe at 360?
u8	2	776 B	556 B	416 B	Yes
int16	4	1496 B	1076 B	776 B	Yes (4 B margin)
int24	8	2936 B	2076 B	1496 B	No — must reduce pairs
float32	8	2936 B	2076 B	1496 B	No — must reduce pairs

For VITA-A/1.0-TX add 4 bytes for the trailer and subtract 4 for the missing integer timestamp: sizes are identical.

NOTE	For int24 and float32 encoding, the device SHALL reduce <code>samples_per_packet</code> so that the IP datagram stays at or below 1500 bytes. The maximum for int24 and float32 is 180 pairs. The actual <code>samples_per_packet</code> is returned in the SLC-CP <code>open_stream</code> response; clients SHALL use this value for all calculations.
-------------	--

Appendix B. Profile Comparison

Feature	VITA-A/1.0 (IF Data)	VITA-A/1.0-CTX (Context)	VITA-A/1.0-TX (Transmit)
Packet Type	0x1	0x4	0x1
Class Identifier	OUI 0x402814, PCC 0x0001	Absent	OUI 0x402814, PCC 0x0002
TSI / TSF	01b / 10b (11b / 10b relative)	00b / 00b	00b / 01b
Trailer	No	No	Yes: burst delimiters
CIFO	—	0x01008000 (+ bit 31 on change)	—
Reference Level	—	Q9.7 dBm in bits 15:0	—
Payload Format	—	Per negotiated encoding (§7.5)	—
Packet size	7 + N words	6 words (24 bytes)	6 + N + 1 words
Emitted when	Continuously during active stream	Stream start; gain change; mode change; optionally periodic	Continuously, or per burst
Tier	RX-T1	RX-T2	Accepted TX-T1 (plain), TX-T2 (bursts)

Appendix C. Implementation Checklist

Tier 1 IF Data packet sender:

- VRT Header constant 0x18600000 (type=1, CID=1, TSI=01b, TSF=10b, no trailer); 0x18E00000 if TSI=11b.
- Packet Count maintained as modulo-16 counter per stream.
- Packet Size field set to 7 + payload words.
- Stream Identifier set from `vita.stream_id` in the `open_stream` response.
- Class ID Word 0 = 0x00402814; Word 1 = 0x00000001.
- Timestamp Integer from the best available clock; `sample_count` reset to 0 on each `start_rx`.
- Timestamp Fractional computed using integer arithmetic from the nominal rate.
- I/Q payload big-endian, interleaved, whole words.
- IP datagram size $\leq$ 1500 bytes.
- UDP sent to `vita.destination` from the `open_stream` request.
- Stream stops immediately on TCP control connection loss.

Tier 2, adds Context packet sender:

- Context packet counter (separate from IF Data counter) initialised to 0 at stream open.
- Context packet sent before first IF Data packet after `start_rx`, after any `direct_sampling` change, after any gain change.
- VRT Header for Context: 0x40000006 | (count $\ll$ 16).
- CIFO = 0x01008000, with bit 31 set on change.
- Reference Level Q9.7 in bits 15:0; bits 31:16 zero.
- Payload Format Word 0 from the §7.5 table for the negotiated encoding and current mode; Word 1 = 0.
- `current_reference_level_dbm` updated on each gain change: `new = old - gain_delta_dB`.
- AGC gain-change handler emits Context packet, rate-limited to 1 per 100 ms.

Transmit, Tier 1 and Tier 2:

- Bind `listen_port`; validate OUI/PCC; accept PCC 0x0001 as continuous transmission.
- Read pair count from the Packet Size field.
- Data timeout enforced from the last packet received.
- [T2] PCC 0x0002: honour trailer BURST_START/BURST_END with enable bits; schedule on the TX sample counter; drop late bursts with `tx_burst_late`.
- [T2] `get_timestamps` implemented with `capture_spread_ns` measured.